Advanced Machine Learning, ETH Zurich

Transformers

From one attention layer to the transformer

Carlos Cotrini

Autumn semester 2026

Entry 1

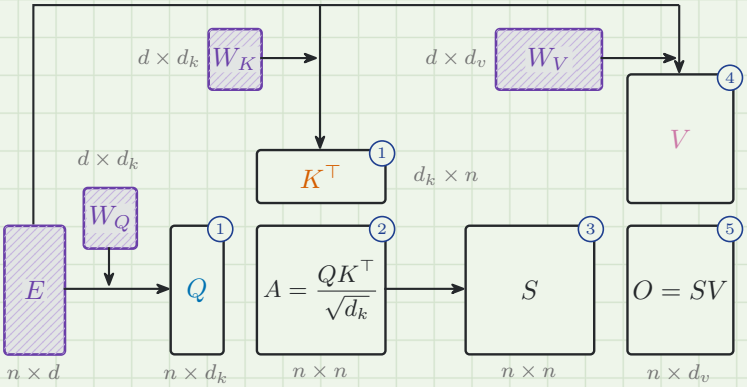
Multi-head attention



So far. The network of the language model deck reads the last 11 words through plain layers. The attention deck gave every word a way to read every other word: a query, a key and a value, the scores A, the weights S and the new rows $O = SV$.

Now. Six parts turn one attention layer into a transformer. First, several attention layers side by side: multi-head attention.

One attention layer, as the attention deck drew it: queries, keys and values, the scores A, the weights S and the new rows $O = SV$. Training learns the violet, hatched blocks.

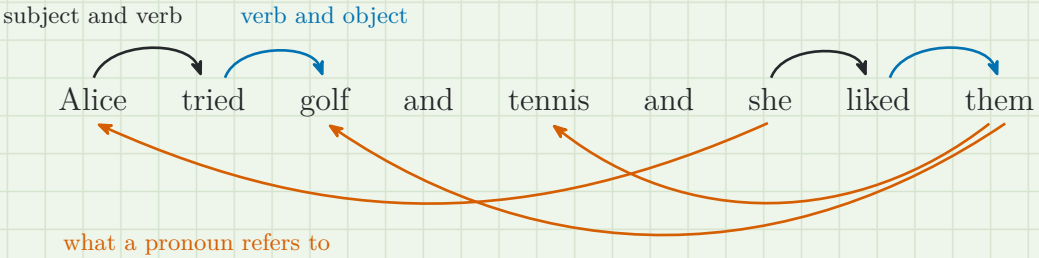


We divide by $\sqrt{d_k}$: it keeps the scores in a range where the softmax works.

row i of S: the softmax of row i of A

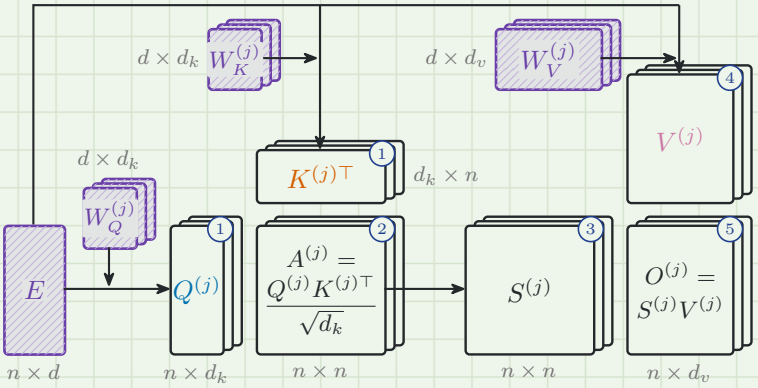
A sentence holds several relations at once

In Wuggish a noun reads the adjectives that fit it: one relation, and one matrix S holds it. An English sentence holds several relations at once.



Each relation draws arrows of its own, so each relation needs a matrix S of its own.

Multi-head attention runs h attention layers side by side on the same rows E: the *heads*. Every block of head j carries the mark (j) above its letter.

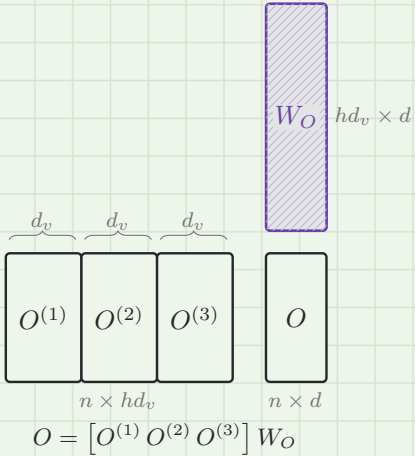


Here $h = 3$. Each head j has its own blocks of weights.

So each head computes its own scores, weights and new rows $O^{(j)}$.

Each head learns its own weights, and so its own $S^{(j)}$: one relation per head.

Each head gives its own new rows $O^{(j)}$. One more block of weights joins them.



Each head j gives n rows of length d_v. Side by side, the h outputs make one block $\left[O^{(1)}\, O^{(2)}\, O^{(3)}\right]$ of n rows of length hd_v.

W_O is a block of weights that training learns: hd_v rows and d columns.

O has one new row of length d for each word: the size of E.

Vaswani et al. (2017) take $h = 8$ heads and $d_v = d/h$.

Entry 2

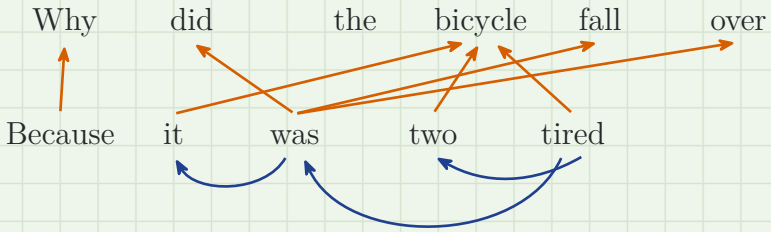
Masked self-attention



So far. Multi-head attention runs h heads, each with its own weights and its own matrix of weights $S^{(j)}$; W_O joins their outputs into n new rows of length d.

Now. A language model writes one word after another, so row i may read only rows 1 to i, as in the language model deck. A mask does this inside attention.

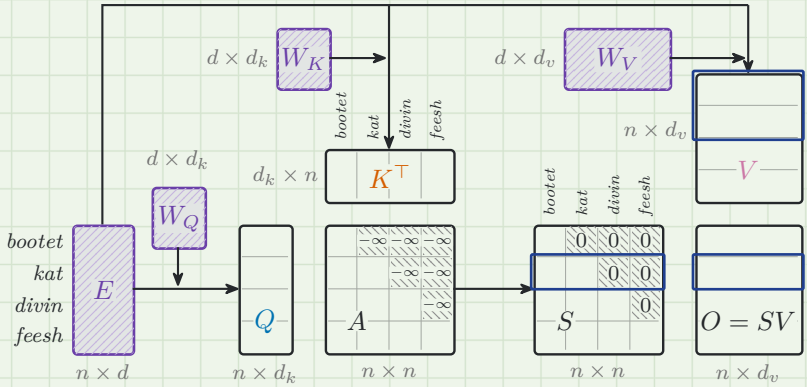
The language model deck ruled that row i may read only its prefix. A model that answers a question writes one word at a time: its prefix is the question and the answer so far.



Each word of the answer reads the question and the answer so far.

Self-attention lets every word read every word. Here each word reads only the words before it, and itself.

Masked self-attention lets word i read only words 1 to i. A *mask* does it: it changes the scores A before the softmax. Here $n = 4$, our sentence of the attention deck.



The mask writes $-\infty$ where a word would read a later word.

$e^{-\infty} = 0$: the softmax gives these cells a weight of 0.

So row i of $O = SV$ adds the values of rows 1 to i only, as row 2, *kat*, shows.

Our sentence puts every adjective before its noun: every 1 of the mall's S sits on or below the diagonal, and the mask keeps them all.

Entry 3

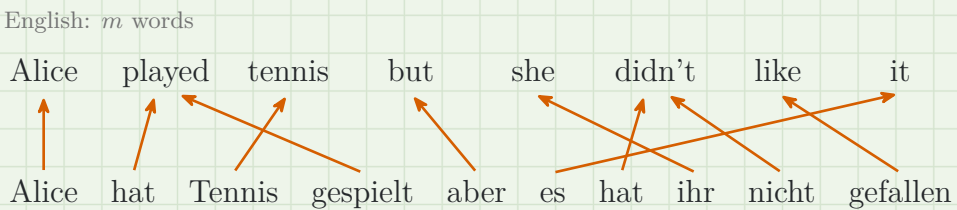
Cross-attention



So far. Masked self-attention: row i reads only rows 1 to i, the words written so far.

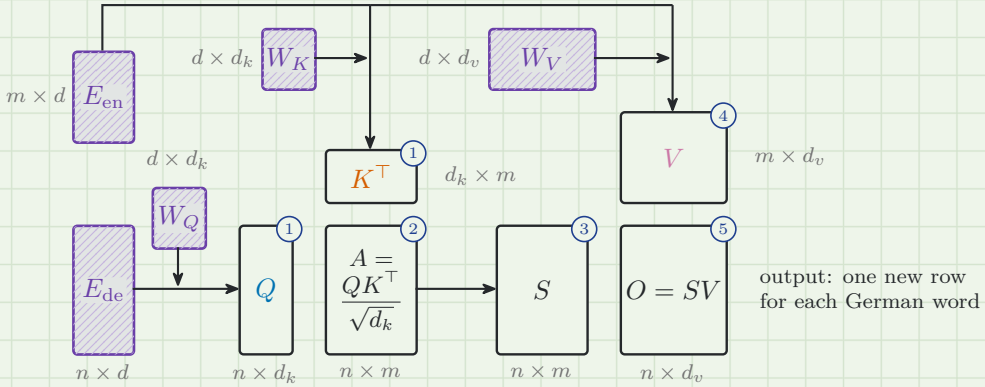
Now. In translation, the sentence being written reads the sentence being translated. Cross-attention takes its queries from one sentence and its keys and values from the other.

Translate an English sentence into German. Each German word, as it is written, reads the English words that it translates.



In self-attention the words of a sentence read each other. Here the words of one sentence read the words of another.

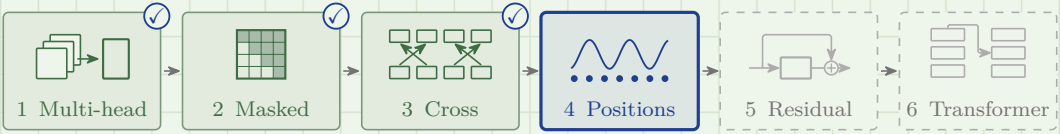
Cross-attention takes the queries from the German sentence, E_{de} with its n rows, and the keys and the values from the English one, E_{en} with its m rows.



Row i of S: the weights of German word i on the m English words.

Entry 4

Positional encodings

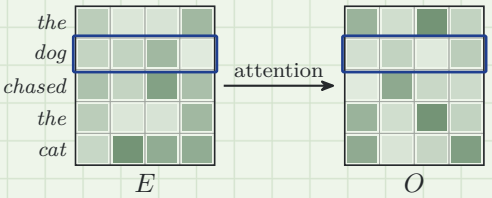


So far. Three kinds of attention: multi-head, masked and cross-attention. Each one compares rows, and its weights come from queries and keys alone.

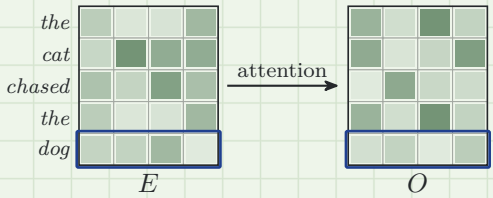
Now. A row says nothing about where its word stands in the sentence. A positional encoding writes the position into every row.

"The dog chased the cat" and "the cat chased the dog" have the same five words, so the same five rows of E, in another order. Run self-attention without a mask on both.

"*the dog chased the cat*"



"*the cat chased the dog*"



Each shade is one number: the darker, the larger.

Every row gives its own query, key and value, so the new rows O move with the rows of E.

So *dog* gets the same new row in both sentences, and so does *cat*: attention cannot tell who chased whom. The two words *the* even get the same new row.

Number the positions from 0 and write each number in binary, its lowest digit first. The code of position i is row i of a block P; its digit j sits in column j.

i		$j=0$	$j=1$	$j=2$
0	*the*	0	0	0
1	*dog*	**1**	0	0
2	*chased*	0	**1**	0
3	*the*	**1**	**1**	0
4	*cat*	0	0	**1**

P

Column 0 flips at every row.

Column 1 flips every 2 rows.

Column 2 flips every 4 rows.

$$P_{ij} = \lfloor i/2^j \rfloor \bmod 2$$

The two words *the* get different rows of P: 000 at position 0 and 110 at position 3.

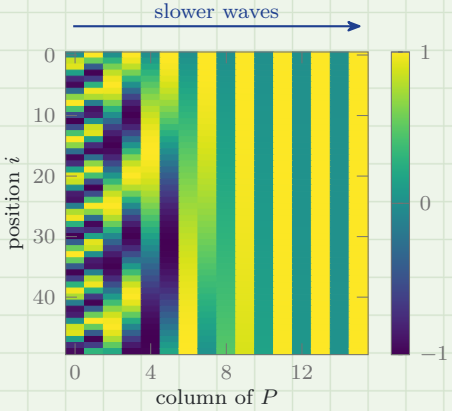
Vaswani et al. (2017) replace the digits by waves: columns $2j$ and $2j+1$ hold the sine and the cosine of the position i, each pair at its own speed.

$$P_{i,2j} = \sin\bigl(i/10000^{2j/d}\bigr)$$

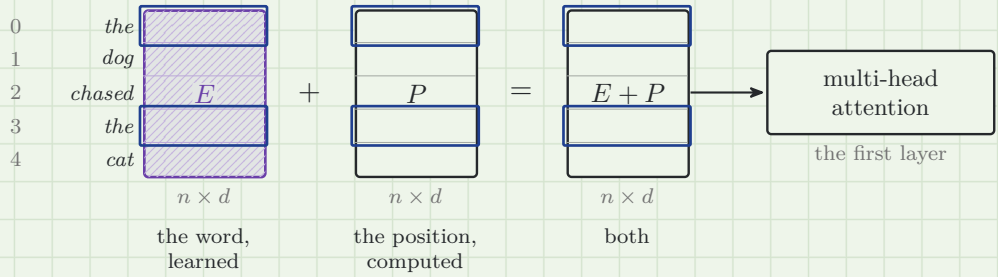
$$P_{i,2j+1} = \cos\bigl(i/10000^{2j/d}\bigr)$$

The heat map: P for 50 positions and $d = 16$, from -1 (dark purple) to 1 (yellow).

As in the binary code, the left columns change fast and the right ones slowly: columns 0 and 1 repeat every 6.3 positions, columns 14 and 15 every 19,869. No two rows are equal.

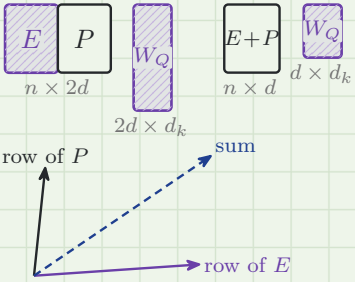


A word now has two rows of length d: its row of E, learned in training, and its row of P, computed from its position. Their sum carries both.



The two words *the* have equal rows of E and different rows of P, so different rows of $E + P$. Attention can now tell *the dog chased the cat* from *the cat chased the dog*.

We could also put the row of P beside the row of E, a row of length $2d$. Three reasons favour the sum.



$$(E + P)\,W_Q = E\,W_Q + P\,W_Q$$

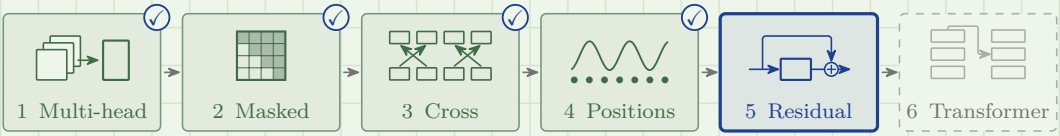
Width. Side by side, every row has length $2d$, so every block of weights that reads it needs $2d$ rows: in Vaswani et al., W_Q of one head grows from 32,768 numbers to 65,536.

Directions. In many dimensions, two rows that have nothing to do with each other meet at nearly a right angle: for $d = 512$, 99 pairs of random rows in 100 meet between 83 and 97 degrees. So the sum keeps both directions.

Linear products. The products that follow act on the two parts separately: every query has a part from the word and a part from the position, and so does every key and value.

Entry 5

Residual blocks



So far. The first layer reads $E + P$: every row carries its word and its position.

Now. A transformer stacks many layers. Residual connections keep a deep stack trainable, and a small network on every row completes a block.

He et al. (2016) trained plain networks, layers stacked one on the other, on small photographs (CIFAR-10): the one with 56 layers erred more than the one with 20.

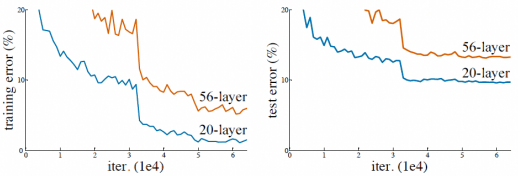


Figure 1. Training error (left) and test error (right) on CIFAR-10 with 20-layer and 56-layer "plain" networks. The deeper network has higher training error, and thus test error. Similar phenomena on ImageNet is presented in Fig. 4.

Figure 1 of Kaiming He, Xiangyu Zhang, Shaoqing Ren and Jian Sun. Deep Residual Learning for Image Recognition. CVPR 2016, pages 770 to 778. arXiv 1512.03385.

On the left, the error on the training data: the deeper network does worse even there, so overfitting cannot explain it.

The cause is mostly that information passes badly through many layers, forward when the network computes and backward when it is trained.

He et al. (2016) let the input of a few layers skip them and added it to their output: a *residual block*. A transformer puts its attention layer in such a block.

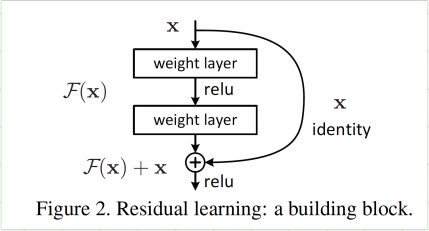
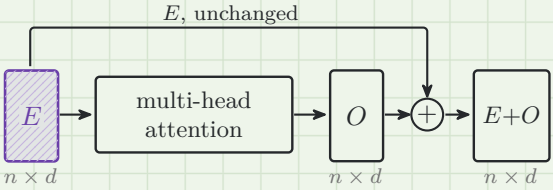


Figure 2. Residual learning: a building block.

Figure 2 of Kaiming He, Xiangyu Zhang, Shaoqing Ren and Jian Sun. Deep Residual Learning for Image Recognition. CVPR 2016, pages 770 to 778. arXiv 1512.03385.

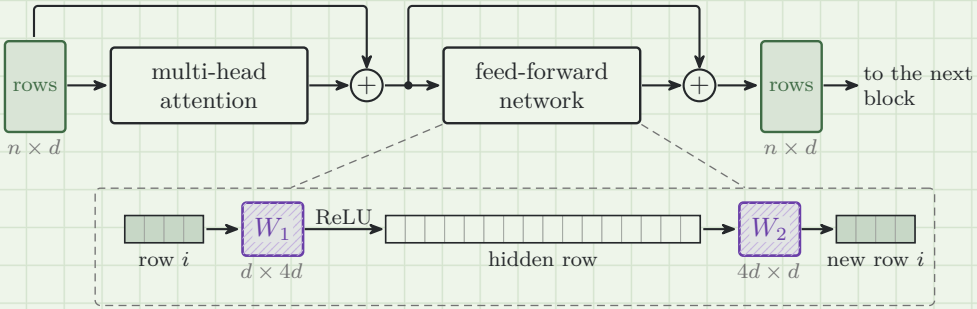
In the figure, the input $\mathbf{x}$ skips two layers and is added to their output $\mathcal{F}(\mathbf{x})$.



Multi-head attention turns E into O, one new row of length d for each word.

So the block outputs $E + O$: each word keeps its row and adds what it found in the others.

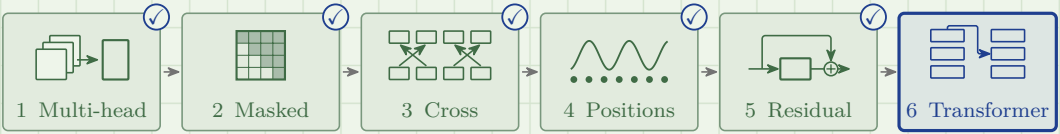
A transformer block has two layers, each in a residual block: multi-head attention, then a small *feed-forward network* that every row passes through on its own.



The same W_1 and W_2 for every row: each row passes through the network on its own. Vaswani et al. take $d = 512$ and a hidden row of 2,048 numbers.

Entry 6

The transformer

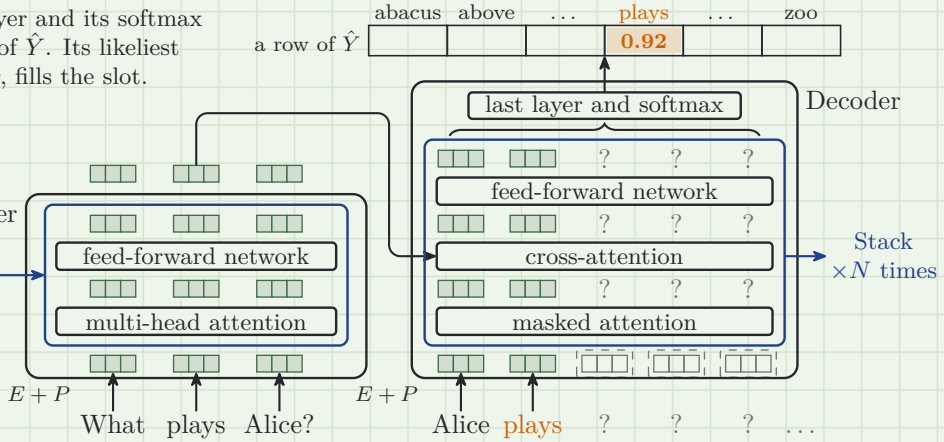


So far. Multi-head, masked and cross-attention; the positions added to the embeddings; blocks with residual connections, stacked.

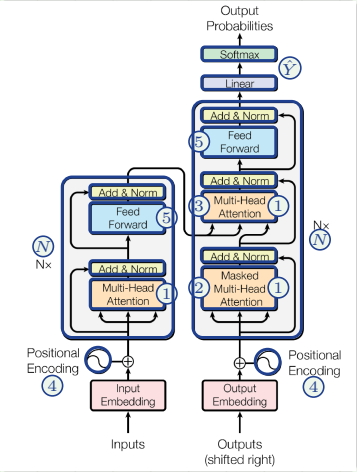
Now. A simplified transformer, then the transformer of Vaswani et al. (2017).

The encoder reads the question; the decoder writes the answer, one word at a time. Every layer adds its input to its output, as in part 5, and each stack repeats N times.

The last layer and its softmax give a row of $\hat{Y}$. Its likeliest word, *plays*, fills the slot.



Their Figure 1 draws the encoder on the left and the decoder on the right.



1. Multi-head attention, in all three orange boxes.
2. Masked attention: row i reads only rows 1 to i.
3. Cross-attention: the decoder's rows read the encoder's rows.
4. Positional encodings: the first layers read $E + P$.
5. Add & Norm: the residual connection, then a rescaling of every row. Feed Forward: the network on every row.

N N×: each stack repeats its block N times.

$\hat{Y}$ Linear, Softmax: the scores and their softmax, a row of $\hat{Y}$.

A language model such as GPT or Llama keeps the right column alone, without its cross-attention.

Figure 1 of Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser and Illia Polosukhin. *Attention Is All You Need.* Advances in Neural Information Processing Systems 30 (NeurIPS 2017). arXiv:1706.03762.